

Collections Framework

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor (CSE), PIT
Parul University

1.	Set Interface and Implementations.....	1
2.	Map Interface and Implementations	2
3.	Summary of Key Concepts.....	4
4.	References.....	5

10.5.1 Set Interface and Implementations

◆ 1. Features of Set Interface

The Set interface is a part of the java.util package and extends the Collection interface.

✓ Key Characteristics:

- **No duplicate elements** allowed.
- **Unordered collection** (except for specific implementations).
- Useful for **unique data storage** like IDs, usernames, etc.

✓ Common Methods:

Method	Description
add(E e)	Adds element if it is not already present
remove(Object o)	Removes specified element
contains(Object o)	Checks if element exists
size()	Returns number of elements
isEmpty()	Checks if set is empty
clear()	Removes all elements

◆ 2. Implementations of Set Interface

✓ a. HashSet

- **Unordered**, uses **hashing**.
- No guarantee of element order.
- Allows **null**.
- Offers **constant-time performance** for basic operations.
- Best for checking **existence** of elements.

```
Set<String> hashSet = new HashSet<>();
hashSet.add("Java");
hashSet.add("Python");
hashSet.add("Java"); // Duplicate, will be ignored
System.out.println(hashSet); // Output may vary in order
```

✓ b. LinkedHashSet

- Maintains **insertion order**.
- Internally uses a **linked list** with hash table.
- Slightly slower than HashSet due to ordering.

```
Set<Integer> linkedSet = new LinkedHashSet<>();
linkedSet.add(10);
linkedSet.add(20);
linkedSet.add(10); // Duplicate ignored
System.out.println(linkedSet); // Output: [10, 20]
```

✓ c. TreeSet

- Implements **SortedSet** interface.
- Maintains elements in **ascending natural order**.
- Does not allow null elements.
- Slower than HashSet, but useful for **sorted data**.

```
Set<String> treeSet = new TreeSet<>();
treeSet.add("Banana");
treeSet.add("Apple");
treeSet.add("Cherry");
System.out.println(treeSet); // Output: [Apple, Banana, Cherry]
```

◆ 3. Behavior Regarding Duplicates and Ordering

Feature	HashSet	LinkedHashSet	TreeSet
Allows Duplicates	✗ No	✗ No	✗ No
Maintains Order	✗ No	✓ Insertion Order	✓ Sorted Order
Null Allowed	✓ One null	✓ One null	✗ No null
Performance	✓ Fastest	⚠ Moderate	⚠ Slowest
Use Case	Unique items	Predictable iteration	Sorted items

10.6.1 Map Interface and Implementations

◆ 1. Map Interface in Java

The Map interface (in java.util) represents a collection that maps **keys to values**.

✓ Key Features:

- Stores data as **key-value pairs**.
- **Unique keys**, but **duplicate values** allowed.
- Each key maps to **at most one value**.

✓ Common Methods:

Method	Description
put(K key, V value)	Adds or updates a key-value pair
get(Object key)	Returns the value for the key
remove(Object key)	Deletes the entry for the key
containsKey(Object key)	Checks if a key exists
containsValue(Object value)	Checks if a value exists
keySet()	Returns a set of keys
entrySet()	Returns a set of key-value pairs
values()	Returns a collection of values

◆ 2. Map Implementations

✓ a. HashMap

- **Unordered** key-value map.
- Allows **one null key** and **multiple null values**.
- Best performance for insertion and lookup.
- Not synchronized.

```
Map<String, Integer> hashMap = new HashMap<>();
hashMap.put("Apple", 10);
hashMap.put("Banana", 5);
System.out.println(hashMap); // Order not guaranteed
```

✓ b. LinkedHashMap

- Maintains **insertion order**.
- Slightly slower than HashMap.
- Allows one null key.
- Useful when predictable iteration is needed.

```
Map<String, String> linkedMap = new LinkedHashMap<>();
linkedMap.put("Name", "John");
linkedMap.put("Role", "Developer");
System.out.println(linkedMap); // Output in insertion order
```

✓ c. TreeMap

- Maintains **ascending sorted order** based on keys.
- Does **not allow null keys**.
- Implements NavigableMap and SortedMap.

```
Map<Integer, String> treeMap = new TreeMap<>();  
treeMap.put(3, "Three");  
treeMap.put(1, "One");  
treeMap.put(2, "Two");  
System.out.println(treeMap); // Output: {1=One, 2=Two, 3=Three}
```

◆ 3. Traversing and Modifying Maps

✓ Using for-each loop with entrySet():

```
(Map.Entry<String, Integer> entry : hashMap.entrySet()) {  
    System.out.println(entry.getKey() + ": " + entry.getValue());  
}
```

✓ Using keySet() to access only keys:

```
for (String key : hashMap.keySet()) {  
    System.out.println("Key: " + key);  
}
```

✓ Modifying a Map:

```
hashMap.put("Apple", 15); // Updates value  
hashMap.remove("Banana"); // Removes key
```

➤ Summary of Key Concepts:

- Set is used for storing **unique elements**.
- HashSet is **fast** but **unordered**.
- LinkedHashSet maintains **insertion order**.
- TreeSet keeps elements **sorted** but is **slower**.
- Map stores data as **key-value pairs**.
- HashMap is **unordered** and **fast**.
- LinkedHashMap maintains **insertion order**.
- TreeMap maintains **sorted key order**.
- You can traverse maps using `entrySet()`, `keySet()`, and `values()`.

➤ References:

- Oracle Docs – Set Interface
 <https://docs.oracle.com/javase/8/docs/api/java/util/Set.html>
- GeeksforGeeks – Set Interface in Java
 <https://www.geeksforgeeks.org/set-in-java/>
- Baeldung – Set Implementations in Java
 <https://www.baeldung.com/java-set>
- Oracle Java Documentation – Map Interface
 <https://docs.oracle.com/javase/8/docs/api/java/util/Map.html>
- GeeksforGeeks – Map Interface in Java
 <https://www.geeksforgeeks.org/map-interface-java-examples/>
- Baeldung – Java Map Interface and Implementations
 <https://www.baeldung.com/java-map>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | GRADE



    
<https://paruluniversity.ac.in/>